

EST 613: Códigos do Módulo I

Camila Ferreira Azevedo

2026-08-24

Contents

Descrição	2
Introdução	2
Exercício 1: Cálculo de probabilidade por meio de simulação	3
Passo 1: Probabilidade de teste positivo	3
Passo 2: Aplicar Teorema de Bayes	3
Exercício 2: Otimização de probabilidade	5
Exercício 3: Teorema de Bayes em problemas de busca e localização	7
Parte 1: Definição das probabilidades	7
Parte 2: Atualização das probabilidades após busca negativa no setor 13	8
Parte 3: Atualização após busca negativa em dois setores, 13 e 12	10
Entendimento da atualização	12
Influência do parâmetro θ	13
Exercício 4: Distribuição Binomial	14
a) Distribuição de probabilidade	15
b) Função de probabilidade	15
c) Distribuição acumulada	15
d) Gráficos	15
e) Quantil	16
f) Geração de valores simulados e histograma	16
g) Medidas de posição e dispersão	17

Exercício 5: Distribuição exponencial	18
Parâmetro taxa	19
Parâmetro escala	19
a) Função densidade de probabilidade	19
b) Função de distribuição acumulada	19
c) Gráficos	20
d) Quantil	20
e) Geração de valores simulados e histograma	21
f) Medidas de posição e dispersão	21
Exercício 6	22
Parâmetro de forma	22
Gráficos	23
Cálculo da Moda	23
Procedimento 1	23
Procedimento 2	24
Efeito dos parâmetros	25
Exercício 7	26
Passo 1: Ajuste das distribuições	26
Passo 2: Gráfico do histograma com as densidades ajustadas	28
Passo 3: Testes de aderência (Kolmogorov-Smirnov)	29
Passo 4: Simulação e visualização de ajustes exponenciais para diferentes tamanhos amostrais	29

Descrição

Este documento apresenta exercícios resolvidos do Módulo I de EST 613, abordando probabilidade, variáveis aleatórias e distribuições de probabilidade, sob uma perspectiva computacional, explorando exemplos e simulações para facilitar a compreensão.

Introdução

No software R, é possível gerar valores de uma determinada distribuição de probabilidade, o que facilita o cálculo das medidas de posição e dispersão, além de proporcionar um melhor entendimento do comportamento das distribuições de probabilidade. Também é possível calcular valores de probabilidade e densidade, quantis (percentis) e probabilidades por meio da função acumulada.

As funções no R, responsáveis por esses cálculos, são denominadas e organizadas da seguinte forma:

`ldist(argumentos)`

- **l**: letra referente ao tipo de cálculo desejado;

- **dist**: sigla da distribuição de probabilidade desejada;
- **argumentos**: argumentos da função e valores dos parâmetros da distribuição de probabilidade desejada.

A **l** deve ser substituída por uma das opções abaixo:

- **r** – gera números aleatórios baseados na distribuição de probabilidade desejada (ex: **rnorm**, **rbinom**, etc.)
- **p** – calcula o valor da função acumulada em um determinado ponto;
- **q** – calcula o valor de **x** correspondente à probabilidade acumulada **p**. Calcula quantis (percentis) a partir da inversa da função acumulada;
- **d** – calcula valores das funções de probabilidade (caso discreto) ou funções densidade de probabilidade (caso contínuo).

Exercício 1: Cálculo de probabilidade por meio de simulação

Um laboratório testa a presença de uma doença em uma população com diferentes taxas de prevalência.

Suponha que: - A probabilidade de a pessoa estar doente é 5%. - O teste tem 95% de sensibilidade (verdadeiro positivo) e 90% de especificidade (verdadeiro negativo).

Pergunta: Qual a probabilidade de uma pessoa estar doente, dado que o teste deu positivo? (Simularemos 1000 testes, como se tivessem sido observados no laboratório, em R, e compararemos o resultado estimado com o valor teórico.)

Dados:

- Prevalência da doença: $P(D) = 0,05$
- Sensibilidade: $P(T+ | D) = 0,95$
- Especificidade: $P(T- | \bar{D}) = 0,90$

Resultado teórico:

Passo 1: Probabilidade de teste positivo

$$P(T+) = P(T+ | D)P(D) + P(T+ | \bar{D})P(\bar{D})$$

$$P(T+) = 0,95 \times 0,05 + (1 - 0,90) \times (1 - 0,05) = 0,1425$$

Passo 2: Aplicar Teorema de Bayes

$$P(D | T+) = \frac{P(T+ | D)P(D)}{P(T+)} = \frac{0,0475}{0,1425} = 0,3333$$

Interpretação: mesmo com teste positivo, há cerca de 33% de chance de a pessoa realmente estar doente, devido à baixa prevalência.

```

# Pacotes necessários
library(animation)

set.seed(123)

# Parâmetros
n <- 1000
prevalencia <- 0.05
sensibilidade <- 0.95
especificidade <- 0.90

# Simular doença
doente <- rbinom(n, 1, prevalencia)
print(doente[1:20])

## [1] 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1

# Simular teste
teste <- ifelse(doente == 1,
               rbinom(n, 1, sensibilidade),
               rbinom(n, 1, 1 - especificidade))
print(teste[1:20])

## [1] 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 1

# Calcular cumulativos
cum_positivos <- cumsum(teste)
print(cum_positivos[1:20])

## [1] 0 0 0 0 0 0 0 0 0 0 1 1 1 1 2 2 2 2 2 3

cum_doentes_pos <- cumsum(teste & doente == 1)
print(cum_doentes_pos[1:20])

## [1] 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 2

# Calcular P(D|T+) acumulado
P_D_dado_T <- ifelse(cum_positivos > 0, cum_doentes_pos / cum_positivos, 0)
print(P_D_dado_T[1:20])

## [1] 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
## [8] 0.0000000 0.0000000 0.0000000 1.0000000 1.0000000 1.0000000 1.0000000
## [15] 0.5000000 0.5000000 0.5000000 0.5000000 0.5000000 0.6666667

# Criar animação
# Observação: o pacote `animation` já captura um frame a cada chamada de plot(),
# então não é necessário usar Sys.sleep() dentro do loop - isso só deixa o knitr
# mais lento (aqui, ~10s de espera artificial em 1000 iterações). O sleep foi
# removido; a velocidade de exibição do GIF é controlada pelo argumento `interval`.
saveGIF({

```

```

par(mfrow=c(1,1))
for (i in 1:n) {
  plot(1:i, P_D_dado_T[1:i], type='l', col='blue', ylim=c(0,1),
       xlab='Número de testes', ylab='P(D|T+)',
       main='Atualização de P(D|T+) a cada teste positivo')
  abline(h = 0.333, col='red', lty=2) # valor teórico de Bayes
  points(i, P_D_dado_T[i], col='blue', pch=16)
}
}, movie.name = "evolucao_probabilidades_doenca.gif", interval = 0.01, ani.width=600, ani.height=400)

## [1] TRUE

```

Exercício 2: Otimização de probabilidade

Uma fábrica quer lançar um novo produto em 5 regiões diferentes, mas não sabe exatamente a aceitação do público em cada região.

- Baseando-se em pesquisas de mercado, a probabilidade de aceitação em cada região é modelada por uma distribuição Beta:

$$p_i \sim \text{Beta}(\alpha_i, \beta_i), \quad i = 1, 2, \dots, 5$$

- O retorno líquido esperado em cada região é dado por:

$$R_i = p_i \times \text{lucro} - \text{investimento inicial}$$

- O objetivo é usar simulação de probabilidade para identificar a região com maior retorno esperado, considerando a incerteza na aceitação do público.

```

set.seed(123)

# Número de regiões
nreg <- 5

# Parâmetros Beta para cada região (exemplo baseado em pesquisa prévia)
alpha <- c(8, 5, 10, 3, 6)
beta <- c(2, 5, 5, 7, 4)

# Lucro por cliente e investimento inicial por região
lucro <- c(10000, 10000, 10000, 10000, 10000) # em R$
investimento <- c(4000, 3500, 5000, 3000, 4500)

# Número de simulações
nsim <- 10000

# Simular aceitação em cada região
# Suponha que a aceitação em cada região tenha sido estimada a partir de dados históricos
# Por exemplo, ao longo de anos anteriores de vendas ou pesquisas de mercado
sim_aceitacao <- matrix(NA, nrow=nsim, ncol=nreg)
for(i in 1:nreg){
  sim_aceitacao[,i] <- rbeta(nsim, alpha[i], beta[i])
}
print(head(sim_aceitacao))

```

```
##           [,1]      [,2]      [,3]      [,4]      [,5]
## [1,] 0.8736191 0.4670971 0.7063867 0.5485151 0.6390356
## [2,] 0.8331787 0.3332576 0.8169667 0.2200335 0.4929365
## [3,] 0.4309544 0.6294669 0.5477518 0.2265472 0.4169264
## [4,] 0.7889204 0.6575098 0.5368859 0.2098582 0.5916705
## [5,] 0.7793337 0.6746209 0.5386399 0.2398565 0.6704030
## [6,] 0.7187909 0.2607897 0.5905982 0.2338518 0.6708656
```

```
# Calcular retorno líquido esperado em cada simulação
sim_retorno <- sweep(sim_aceitacao, 2, lucro, "*") - matrix(investimento, nrow=nsim,
                                                           ncol=nreg, byrow=TRUE)
print(head(sim_retorno))
```

```
##           [,1]      [,2]      [,3]      [,4]      [,5]
## [1,] 4736.1905 1170.9711 2063.8674 2485.1506 1890.3557
## [2,] 4331.7872 -167.4239 3169.6674 -799.6645 429.3654
## [3,] 309.5439 2794.6689 477.5180 -734.5279 -330.7355
## [4,] 3889.2035 3075.0981 368.8588 -901.4184 1416.7051
## [5,] 3793.3375 3246.2093 386.3993 -601.4346 2204.0303
## [6,] 3187.9086 -892.1032 905.9817 -661.4822 2208.6560
```

```
# Calcular retorno médio esperado por região
retorno_medio <- colMeans(sim_retorno)
```

```
# Melhor região
melhor_regiao <- which.max(retorno_medio)
```

```
# Resultados
print("Retorno médio esperado por região:")
```

```
## [1] "Retorno médio esperado por região:"
```

```
print(round(retorno_medio,2))
```

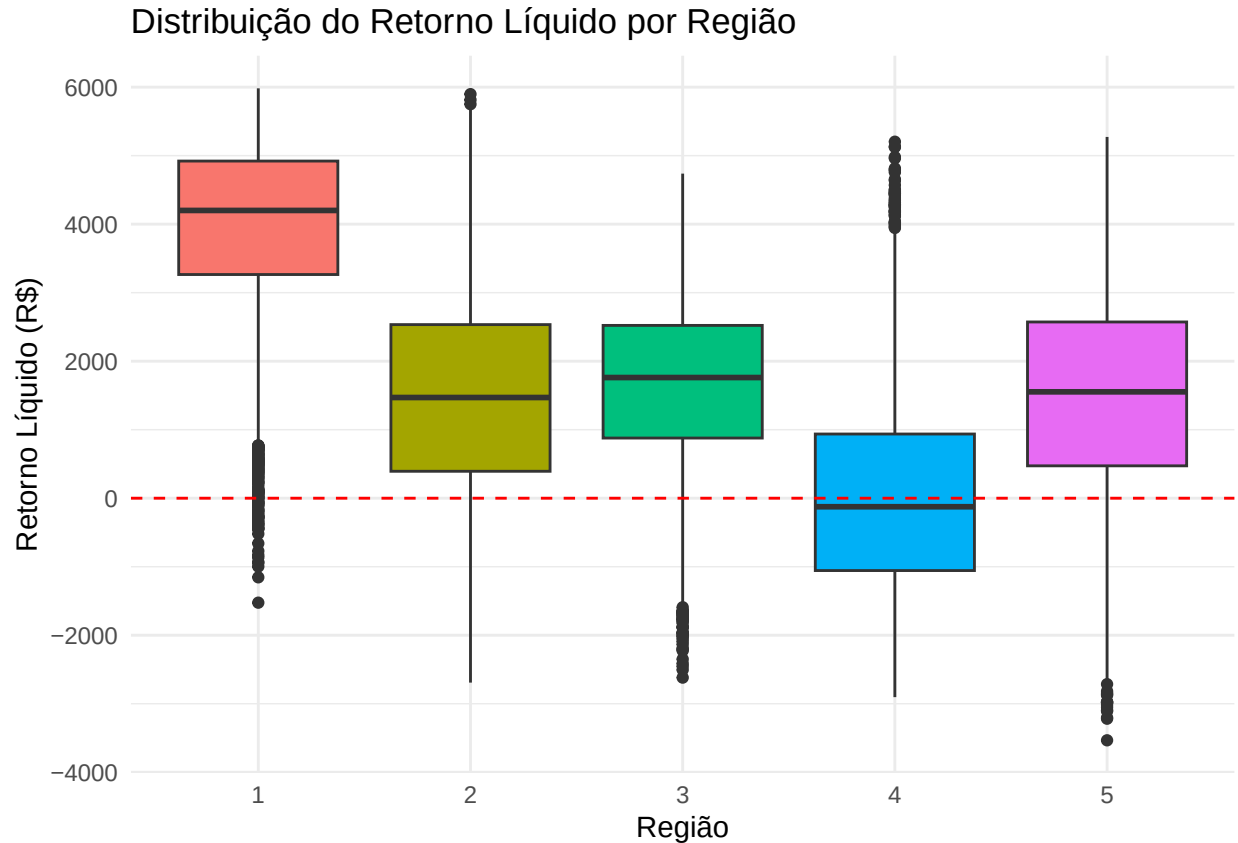
```
## [1] 3994.28 1473.76 1669.40 4.53 1479.53
```

```
cat("Melhor região para lançar o produto:", melhor_regiao, "\n")
```

```
## Melhor região para lançar o produto: 1
```

```
# Visualização
library(ggplot2)
df <- data.frame(Regiao = factor(rep(1:nreg, each=nsim)),
                 Retorno = as.vector(sim_retorno))

ggplot(df, aes(x=Regiao, y=Retorno, fill=Regiao)) +
  geom_boxplot() +
  geom_hline(yintercept=0, linetype="dashed", color="red") +
  labs(title="Distribuição do Retorno Líquido por Região",
       y="Retorno Líquido (R$)", x="Região") +
  theme_minimal() +
  theme(legend.position = "none")
```



Exercício 3: Teorema de Bayes em problemas de busca e localização

O submarino nuclear USS Scorpion desapareceu em 22 de maio de 1968 nas proximidades do arquipélago dos Açores. Após vários dias sem contato, iniciou-se a busca pelo submarino na região do último ponto conhecido. Para facilitar a análise, a área foi dividida em 25 setores (arranjados em uma grade 5x5). Com base em informações disponíveis e avaliações de especialistas, atribuiu-se uma probabilidade inicial $P(S_i)$ para cada setor i de conter o submarino.

Considere que a probabilidade de detectar o submarino, dado que ele está presente no setor procurado, é baixa ($\theta = 0,1$) devido às condições do fundo do mar e às limitações do equipamento de busca. Consideramos que não há falsos positivos, ou seja, não é possível detectar o submarino em um setor onde ele não está.

Parte 1: Definição das probabilidades

Seja S_i o evento “submarino está no setor i ”.

Seja D_i o evento “detecta o submarino no setor i ” e $\overline{D}_i$ “não detecta no setor i ”.

Sabemos:

- $P(S_i)$ = probabilidade a priori do submarino estar no setor
- $P(D_i | S_i) = \theta = 0,1$
- $P(\overline{D}_i | S_i) = 1 - \theta = 0,9$

- $P(D_i | S_j) = 0$ se $j \neq i$ (sem falso positivo)
- $P(\overline{D}_i | S_j) = 1$ se $j \neq i$ (não detecta onde não está)

Problemas a resolver:

- Considerando a distribuição inicial das probabilidades nos 25 setores, atualize as probabilidades de que o submarino esteja em cada setor após uma busca no setor 13 que não resultou em detecção.

Parte 2: Atualização das probabilidades após busca negativa no setor 13

Você fez a busca no setor 13 e não detectou o submarino, então o evento observado é $\overline{D}_{13}$.

Queremos atualizar $P(S_i | \overline{D}_{13})$.

Pela regra de Bayes:

$$P(S_i | \overline{D}_{13}) = \frac{P(\overline{D}_{13} | S_i)P(S_i)}{P(\overline{D}_{13})}$$

Onde o denominador é a probabilidade total do evento observado:

$$P(\overline{D}_{13}) = \sum_{j=1}^{25} P(\overline{D}_{13} | S_j)P(S_j)$$

Sabemos que:

- $j = 13$: $P(\overline{D}_{13} | S_{13}) = 1 - \theta = 1 - 0,1 = 0,9$
- $j \neq 13$: $P(\overline{D}_{13} | S_j) = 1$

Logo, o denominador do Teorema de Bayes é dado por:

$$P(\overline{D}_{13}) = (1 - \theta)P(S_{13}) + 1 \times \sum_{j \neq 13} P(S_j)$$

$$P(\overline{D}_{13}) = P(S_{13}) - \theta P(S_{13}) + \sum_{j \neq 13} P(S_j)$$

$$P(\overline{D}_{13}) = 1 - \theta P(S_{13})$$

Já o numerador depende do valor de j :

- se $j = 13$, tem-se: $P(\overline{D}_{13} | S_{13})P(S_{13}) = (1 - \theta)P(S_{13})$
- se $j \neq 13$, tem-se: $P(\overline{D}_{13} | S_j)P(S_j) = 1 \times P(S_j)$

Portanto,

- se $i = 13$, $P(S_{13} | \overline{D}_{13}) = \frac{(1-\theta)P(S_{13})}{1-\theta P(S_{13})}$
- se $i \neq 13$, $P(S_i | \overline{D}_{13}) = \frac{1 \times P(S_i)}{1-\theta P(S_{13})}$

```

set.seed(123)

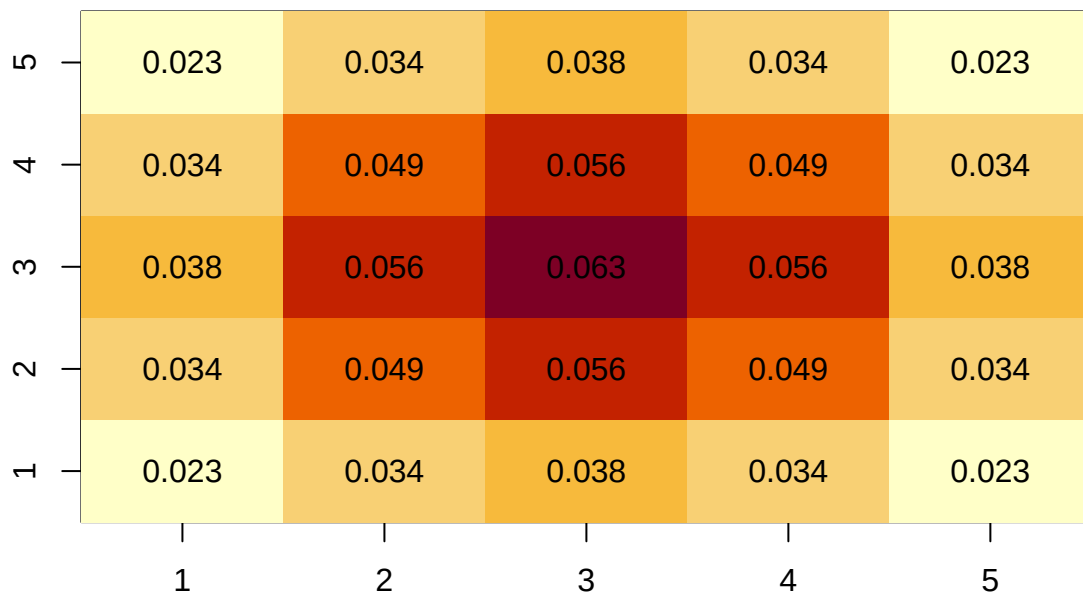
# Definir número de setores (5x5 = 25)
nloc <- 5
seq1 <- 1:nloc
combo <- expand.grid(x=seq1, y=seq1)

# Calcular probabilidades iniciais com normal bivariada (Probabilidades a priori - subjetivas)
combo$prob <- dnorm(combo$x, mean=3, sd=2) * dnorm(combo$y, mean=3, sd=2)
combo$prob <- combo$prob / sum(combo$prob)

# Visualizar probabilidades iniciais
pi1 <- matrix(combo$prob, nloc, nloc, byrow=FALSE)
image(1:nloc, 1:nloc, pi1, xlab='', ylab='',
      main='Probabilidades a priori do submarino estar no setor `i` ')
text(combo$x, combo$y, round(combo$prob, 3), col='black')

```

Probabilidades a priori do submarino estar no setor `i`



```

# Definir probabilidade de detectar dado que está presente
theta <- 0.1

# Função para atualizar probabilidades após busca negativa no setor k
atualiza_prob <- function(probs_apriori, setor_buscado, theta) {
  Pk <- probs_apriori[setor_buscado]
  numerador <- 1 * probs_apriori
  numerador[setor_buscado] <- (1 - theta) * probs_apriori[setor_buscado]
}

```

```

denominador <- sum(numerador)
probs_novas <- numerador / denominador
return(probs_novas)
}

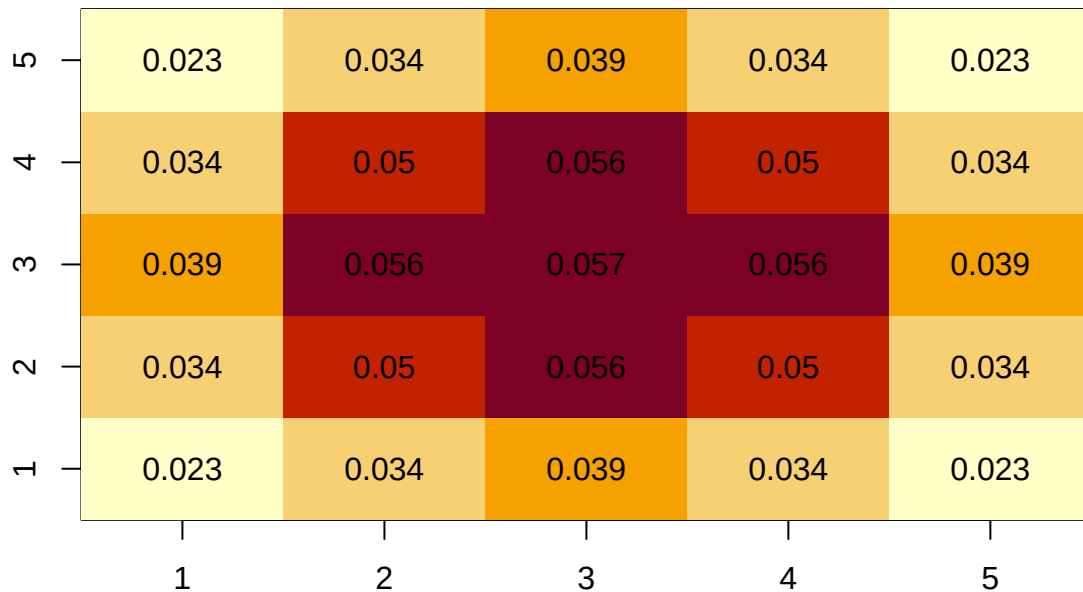
# Probabilidades iniciais como vetor, numeradas de 1 a 25 (linha por linha,
# consistente com a visualização via matrix(..., byrow=TRUE) usada abaixo)
probs <- as.vector(t(pi1))

# Busca no setor 13 (linha 3, coluna 3)
setor1 <- 13
probs1 <- atualiza_prob(probs, setor1, theta)

# Mostrar as probabilidades atualizadas
image(1:nloc, 1:nloc, matrix(probs1, nloc, nloc, byrow=TRUE),
      main='Probabilidades atualizadas após busca no setor 13 (sem detecção)',
      xlab = "", ylab = "")
text(expand.grid(x=1:nloc, y=1:nloc), labels=round(probs1,3))

```

Probabilidades atualizadas após busca no setor 13 (sem detecção)



Parte 3: Atualização após busca negativa em dois setores, 13 e 12

Suponha que após buscar o setor 13, sem encontrar, já atualizamos as probabilidades para $P(S_i | \bar{D}_{13})$. Agora buscamos o setor 12 e também não detectamos.

Como as buscas em setores diferentes são eventos condicionalmente independentes dado S_i (a detecção em

um setor não depende do resultado da busca em outro), podemos aplicar a regra de Bayes **sequencialmente**: usamos $P(S_i | \bar{D}_{13})$ da Parte 2 como a nova probabilidade a priori e atualizamos novamente com o mesmo procedimento, agora condicionando na busca negativa no setor 12:

$$P(S_i | \bar{D}_{13} \cap \bar{D}_{12}) = \frac{P(\bar{D}_{12} | S_i) P(S_i | \bar{D}_{13})}{\sum_{j=1}^{25} P(\bar{D}_{12} | S_j) P(S_j | \bar{D}_{13})}$$

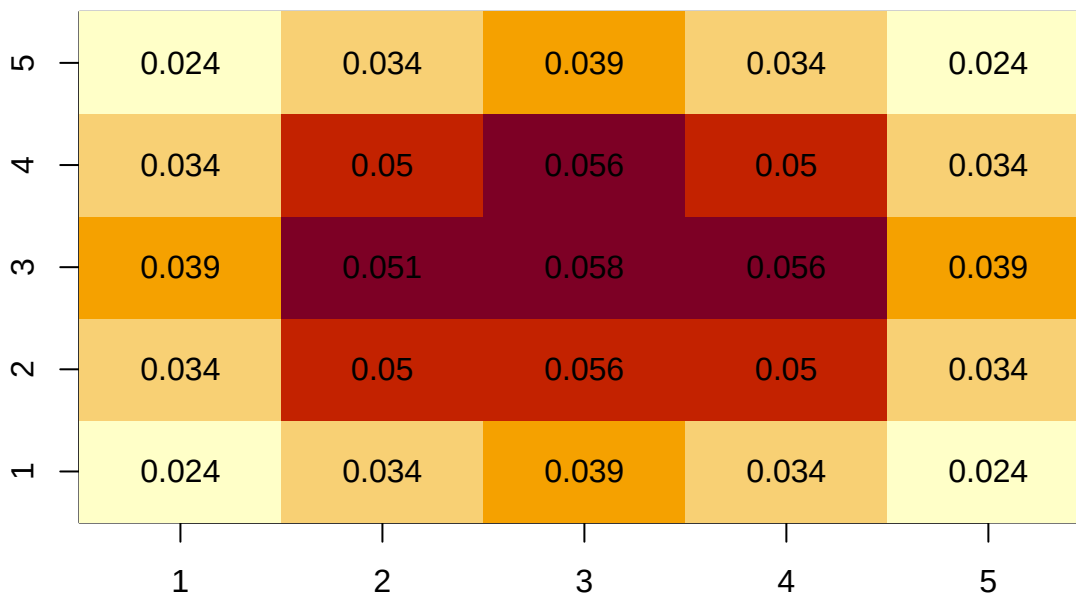
que tem exatamente a mesma forma da fórmula da Parte 2, trocando a priori original $P(S_i)$ pela posteriori já atualizada $P(S_i | \bar{D}_{13})$. É por isso que, no código, basta chamar `atualiza_prob()` novamente, agora passando `probs1` (o resultado da primeira atualização) como entrada.

- Atualize novamente as probabilidades considerando que, após a busca no setor 13 sem sucesso, foi feita uma busca no setor 12 e também não houve detecção.

```
# Busca no setor 12 (linha 3, coluna 2)
setor2 <- 12
probs2 <- atualiza_prob(probs1, setor2, theta)

# Mostrar as probabilidades atualizadas
image(1:nloc, 1:nloc, matrix(probs2, nloc, nloc, byrow=TRUE),
      main='Probabilidades atualizadas após buscas no setores 13 e 12 (sem detecção)',
      xlab = "", ylab = "")
text(expand.grid(x=1:nloc, y=1:nloc), labels=round(probs2,3))
```

Probabilidades atualizadas após buscas no setores 13 e 12 (sem detec



Entendimento da atualização

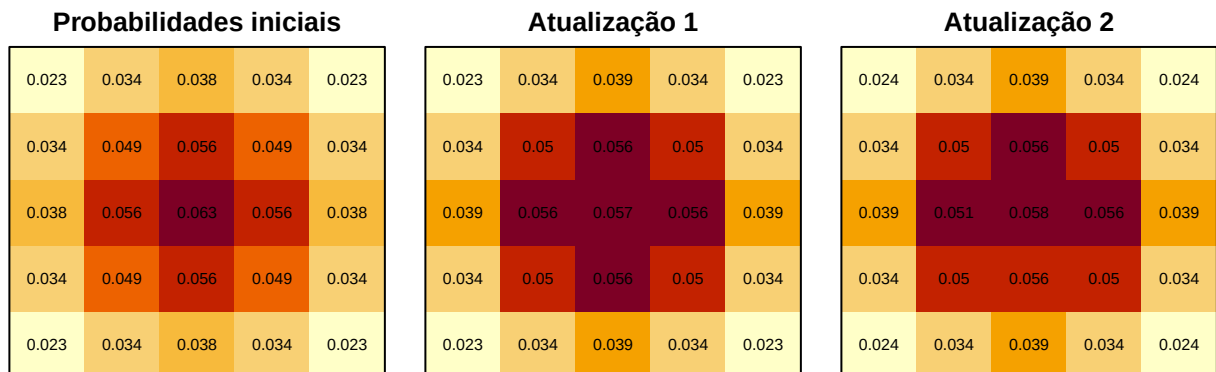
Ao longo das buscas, à medida que o submarino não é detectado em determinados setores, as probabilidades de encontrá-lo em cada setori são atualizadas.

```
# 3 gráficos lado a lado, compactos e "encostados"
layout(matrix(1:3, 1, 3))
par(mar = c(1.2, 1.2, 2, 0.4), xaxs = 'i', yaxs = 'i')

image(1:nloc, 1:nloc, matrix(probs, nloc, nloc, byrow=TRUE),
      main='Probabilidades iniciais', axes=FALSE)
text(expand.grid(x=1:nloc, y=1:nloc), labels=round(probs, 3), cex=0.7)
box()

image(1:nloc, 1:nloc, matrix(probs1, nloc, nloc, byrow=TRUE),
      main='Atualização 1', axes=FALSE)
text(expand.grid(x=1:nloc, y=1:nloc), labels=round(probs1, 3), cex=0.7)
box()

image(1:nloc, 1:nloc, matrix(probs2, nloc, nloc, byrow=TRUE),
      main='Atualização 2', axes=FALSE)
text(expand.grid(x=1:nloc, y=1:nloc), labels=round(probs2, 3), cex=0.7)
box()
```



Para facilitar a visualização das mudanças nos padrões de probabilidade, foi criada uma animação em GIF.

```
library(animation)

# Setores a serem buscados
setores <- c(13, 12, 7, 18, 10, 20, 25) # exemplo
theta <- 0.1

# Criar animação
# (mesmo ajuste do Exercício 1: sem Sys.sleep dentro do loop; a velocidade de
# exibição é controlada pelo argumento `interval` do saveGIF)
saveGIF({
  par(mfrow=c(1,1))
  probs_atual <- probs
  for (i in 1:length(setores)) {
```

```

    image(1:nloc, 1:nloc, matrix(probs_atual, nloc, nloc, byrow=TRUE),
          main=paste("Busca no setor", setores[i]))
    text(expand.grid(x=1:nloc, y=1:nloc), labels=round(probs_atual,3))
    probs_atual <- atualiza_prob(probs_atual, setores[i], theta)
  }
}, movie.name = "evolucao_probabilidades_submarino.gif", interval = 0.5, ani.width=500, ani.height=500)

```

```
## [1] TRUE
```

Influência do parâmetro θ

- Influência do parâmetro θ (probabilidade de detecção dado que o submarino está no setor) na redistribuição das probabilidades após buscas sem sucesso.

É possível estudar e compreender o comportamento das probabilidades à medida que a eficiência de detecção (θ) aumenta ou diminui.

```

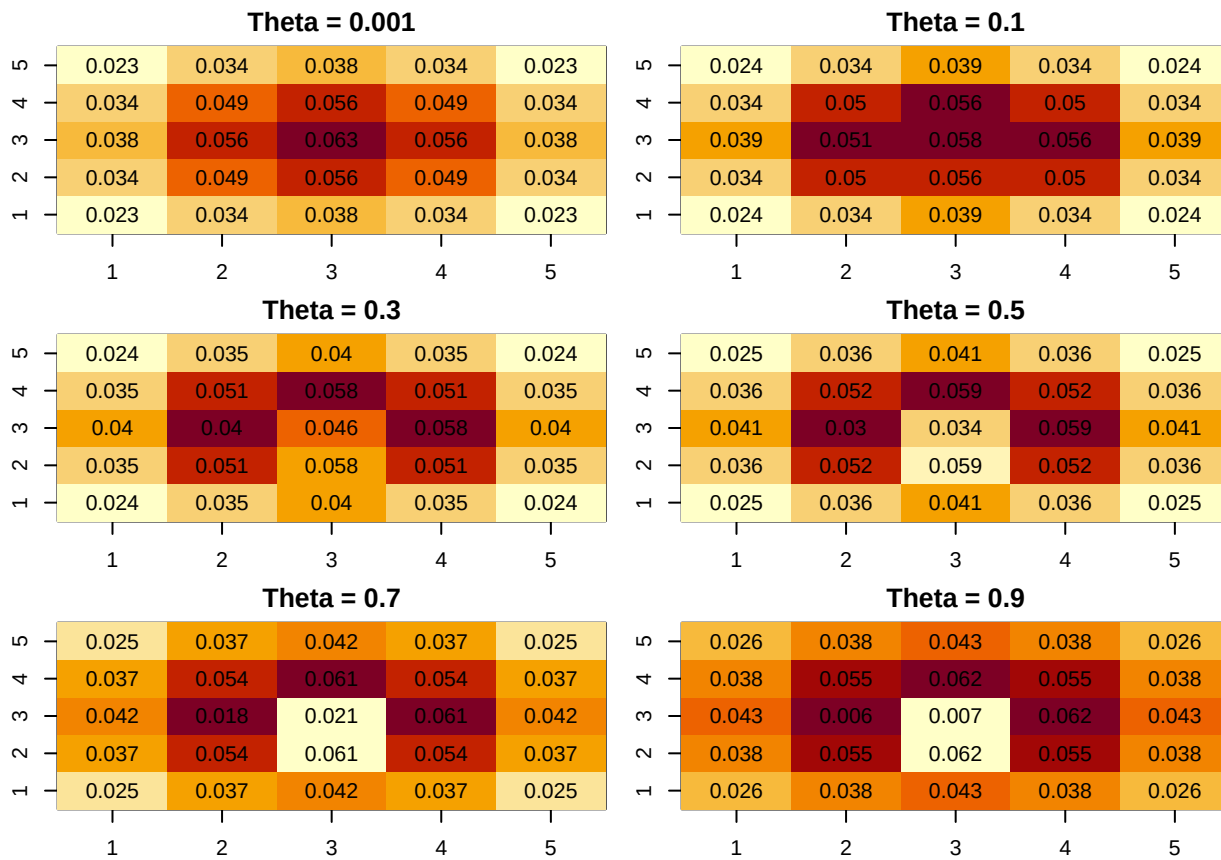
# Diferentes valores de theta para comparar
theta_vals <- c(0.001, 0.1, 0.3, 0.5, 0.7, 0.9)

# Criar lista para armazenar resultados
probs_theta <- list()

for (th in theta_vals) {
  probs1 <- atualiza_prob(probs, setor1, th)
  probs2 <- atualiza_prob(probs1, setor2, th)
  probs_theta[[as.character(th)]] <- probs2
}

# Plot comparativo lado a lado
par(mfrow=c(3,2), mar=c(2,2,2,1))
for (th in names(probs_theta)) {
  image(1:nloc, 1:nloc, matrix(probs_theta[[th]], nloc, nloc, byrow=TRUE),
        main=paste0("Theta = ", th))
  text(expand.grid(x=1:nloc, y=1:nloc), labels=round(probs_theta[[th]], 3))
}

```



Exercício 4: Distribuição Binomial

Considere que:

X representa o número de pacientes imunizados em um grupo de 3 pessoas, sendo a probabilidade de um paciente ser imunizado é igual a 0,90.

$$X \sim \text{Binomial}(3, 0,90)$$

Função de probabilidade:

$$p(x) = \binom{n}{x} p^x (1-p)^{n-x}, \quad x = 0, 1, 2, \dots, n$$

em que n é o número de tentativas e $0 < p < 1$ é a probabilidade de sucesso. No nosso, exemplo $n = 3$ e $p = 0,90$ (probabilidade de um paciente ser imunizado).

O valor esperado e a variância da distribuição binomial são dados por:

$$E(X) = np$$

e

$$V(X) = np(1-p)$$

a) Distribuição de probabilidade

Table 1: Distribuição Binomial com $n = 3$ e $p = 0.9$

Número.de.Sucessos	Probabilidade
0	0.001
1	0.027
2	0.243
3	0.729

b) Função de probabilidade

```
# Parâmetros da distribuição binomial
n = 3
p = 0.90

# Valor da função de probabilidade (P(X = 2))
x = 2
dbinom(x, n, p) # Usando a função do R

## [1] 0.243

choose(n, x) * p^x * (1-p)^(n-x) # Usando a expressão da função de probabilidade

## [1] 0.243
```

c) Distribuição acumulada

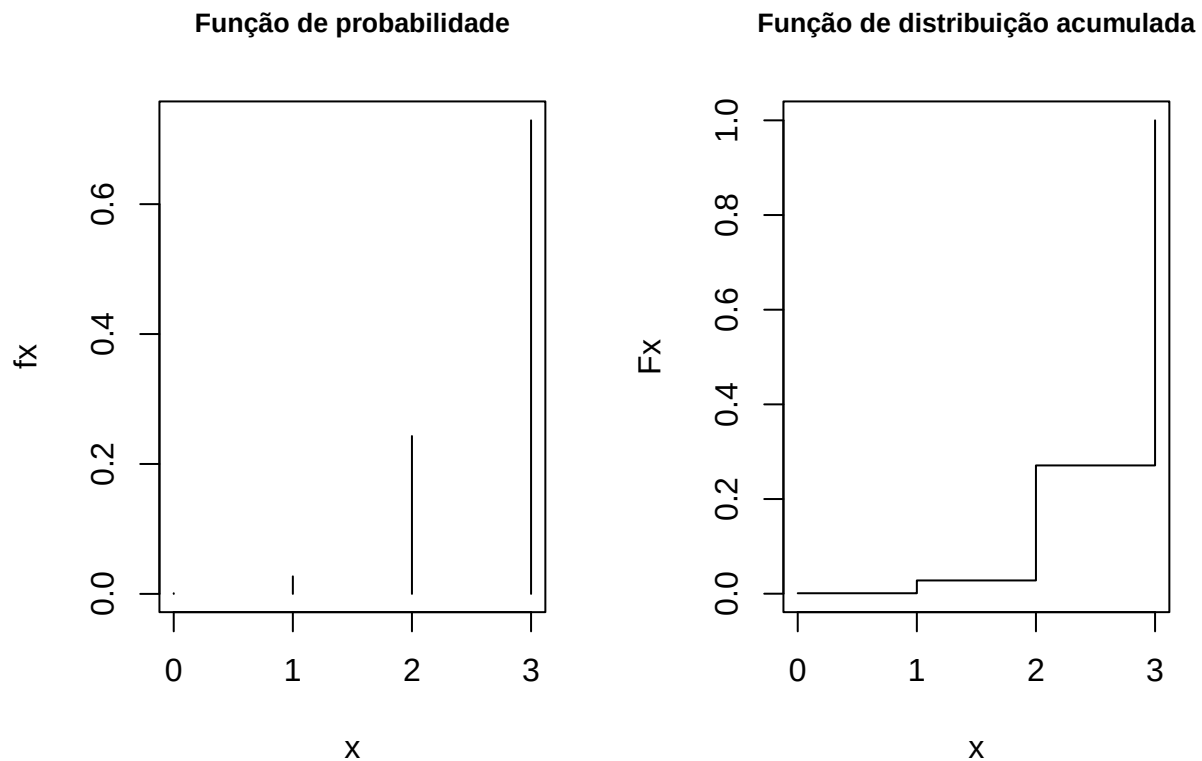
```
# Valor da função de distribuição acumulada até X = 2 (P(X <= 2))
pbinom(x, n, p)

## [1] 0.271
```

d) Gráficos

```
# Gráfico da função de probabilidade e da função de distribuição acumulada
par(mfrow = c(1, 2)) # Configura o layout para exibir dois gráficos lado a lado
x = 0:3 # Valores que X pode assumir
fx = dbinom(x, n, p)
plot(x, fx, type = "h", main = "Função de probabilidade", xaxt = "n", cex.main = 0.8)
axis(side = 1, at = seq(0, 3, by = 1))

Fx = pbinom(x, n, p)
plot(x, Fx, type = "s", main = "Função de distribuição acumulada", xaxt = "n", cex.main = 0.8)
axis(side = 1, at = seq(0, 3, by = 1))
```



e) Quantil

```
# Cálculo do quantil 0,95 da distribuição binomial
qbinom(0.95, n, p)
```

```
## [1] 3
```

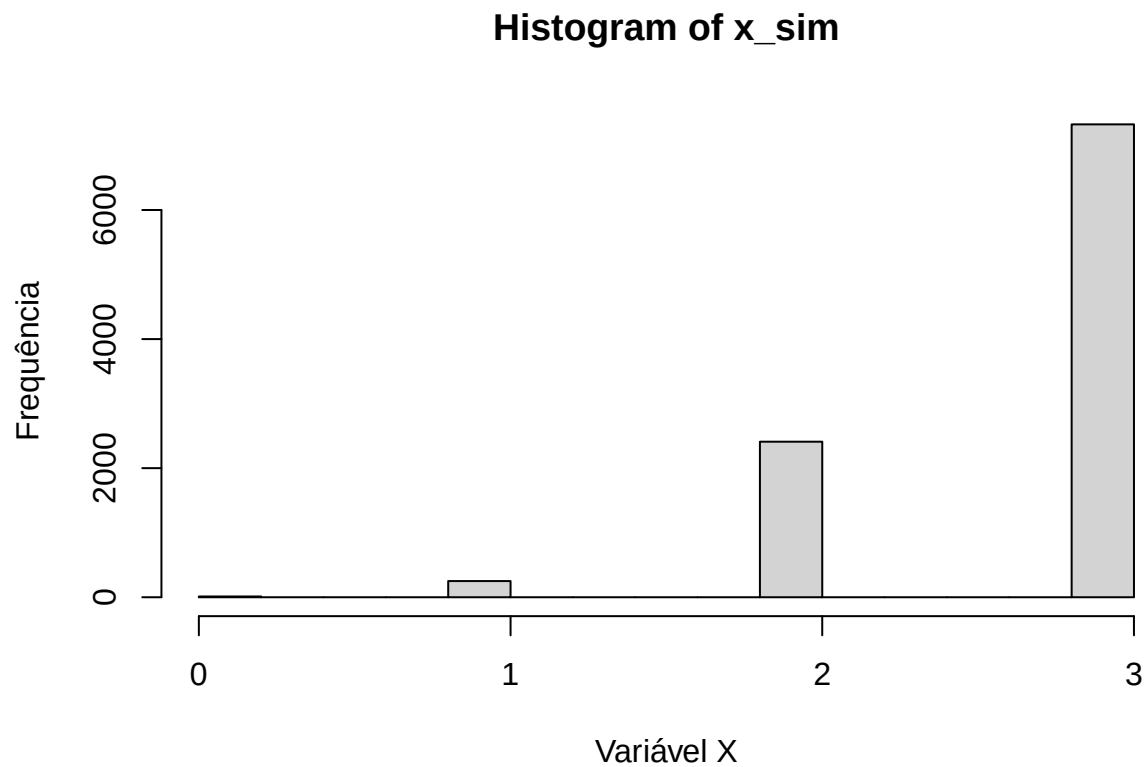
f) Geração de valores simulados e histograma

```
set.seed(123)

# Geração de 10000 valores aleatórios da distribuição binomial com n=3 e p=0,90
x_sim = rbinom(10000, n, p)
x_sim[1:10] # Mostra os primeiros 10 valores gerados
```

```
## [1] 3 2 3 2 2 3 3 2 3 3
```

```
# Construção de histograma dos valores simulados
hist(x_sim, xlab = "Variável X", ylab = "Frequência", freq = TRUE, xaxt = "n")
axis(side = 1, at = seq(0, 3, by = 1))
```



g) Medidas de posição e dispersão

```
# Cálculo da média  
mean(x_sim) # Aproximação numérica
```

```
## [1] 2.7052
```

```
n * p # Valor exato
```

```
## [1] 2.7
```

```
# Cálculo da mediana  
median(x_sim) # Aproximação numérica
```

```
## [1] 3
```

```
qbinom(0.50, n, p) # Valor exato
```

```
## [1] 3
```

```
# Cálculo da moda
table(x_sim) # Valor de maior frequência
```

```
## x_sim
##      0      1      2      3
##    12    251   2410  7327
```

```
# Cálculo da variância
var(x_sim) # Aproximação numérica
```

```
## [1] 0.2653195
```

```
n * p * (1-p) # Valor exato
```

```
## [1] 0.27
```

Exercício 5: Distribuição exponencial

Considere que:

X representa o tempo de vida da lâmpada, medido em horas, e assume-se que o parâmetro de taxa da distribuição seja igual a 0,1.

$$X \sim \text{Exponencial}(0, 10)$$

Função densidade de probabilidade:

$$p(x) = \beta e^{-\beta x}, \quad x > 0$$

em que $\beta > 0$ é o **parâmetro de taxa** da distribuição, que indica a frequência média de ocorrência do evento (nesse caso, falha da lâmpada).

O valor esperado e a variância da distribuição exponencial são dados por:

$$E(X) = \frac{1}{\beta}$$

e

$$V(X) = \frac{1}{\beta^2}$$

Atenção: No R, as funções relacionadas à distribuição exponencial (`dexp`, `pexp`, `qexp`, `rexp`) utilizam como argumento `rate` (parâmetro de taxa β), e não o parâmetro de escala. Sempre confira a parametrização na documentação (`?dexp`) para evitar confusões.

O parâmetro de escala e o parâmetro de taxa mexem no alongamento ou compressão da curva, mas não na forma básica da distribuição — eles controlam quão rápido ou devagar a variável aleatória tende a assumir valores grandes.

Parâmetro taxa

Definição: número médio de eventos (falhas, chegadas, ocorrências) por unidade de tempo.

Unidade: eventos por unidade de tempo.

Interpretação prática:

- $\beta = 0,1$ significa que, em média, ocorrem 0,1 falhas por hora.
- Isso equivale a dizer que leva, em média, 10 horas para ocorrer uma falha.

Parâmetro escala

O parâmetro de escala é o inverso do parâmetro de taxa, ou seja,

$$\theta = \frac{1}{\beta}$$

Definição: tempo médio até a ocorrência do evento.

Unidade: tempo por evento (o inverso da taxa).

Interpretação prática:

- $\theta = 10$ significa que o tempo médio de vida da lâmpada é 10 horas.
- Um valor maior de θ indica eventos mais raros (maior tempo até a ocorrência).

a) Função densidade de probabilidade

```
# Parâmetros da distribuição exponencial
# Atenção para a definição do parâmetro beta. Consulte o help do R se necessário.
# ?rexp - Acessa o help da função rexp no R
beta = 0.1

# Valor da função densidade de probabilidade quando X = 5
x = 5
dexp(x, beta) # Usando a função do R
```

```
## [1] 0.06065307
```

```
beta * exp(-beta * x) # Usando a expressão da função densidade de probabilidade
```

```
## [1] 0.06065307
```

b) Função de distribuição acumulada

```
# Valor da função de distribuição acumulada até X = 5 (P(X <= 5))
pexp(x, beta)
```

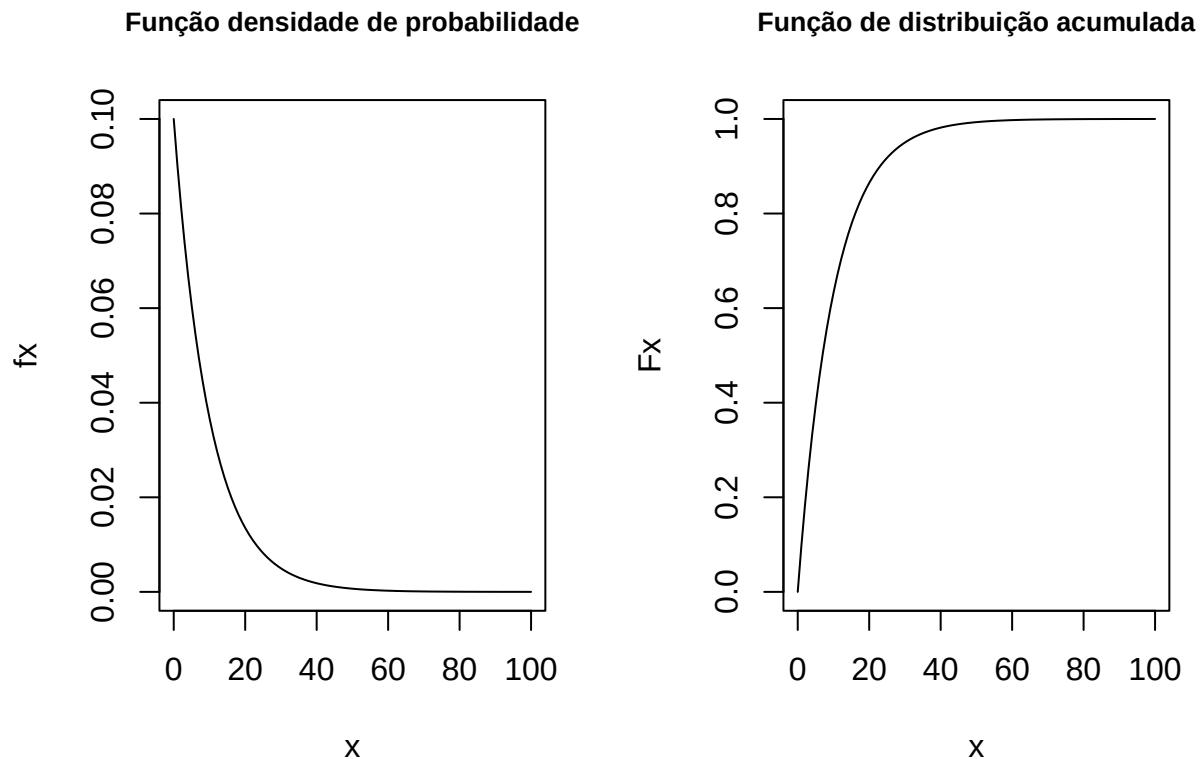
```
## [1] 0.3934693
```

```
# Probabilidade de X assumir valores entre 3 e 5 ( $P(3 < X \leq 5)$ )
pexp(5, beta) - pexp(3, beta)
```

```
## [1] 0.1342876
```

c) Gráficos

```
# Gráfico da função densidade de probabilidade e da função de distribuição acumulada
par(mfrow = c(1, 2))
x <- seq(0, 100, length.out = 100)
fx <- dexp(x, 0.1)
plot(x, fx, type = "l", main = "Função densidade de probabilidade",, cex.main = 0.8)
Fx <- pexp(x, 0.1)
plot(x, Fx, type = "l", main = "Função de distribuição acumulada",, cex.main = 0.8)
```



d) Quantil

```
# Cálculo do quantil 0,95 da distribuição exponencial
qexp(0.95, beta)
```

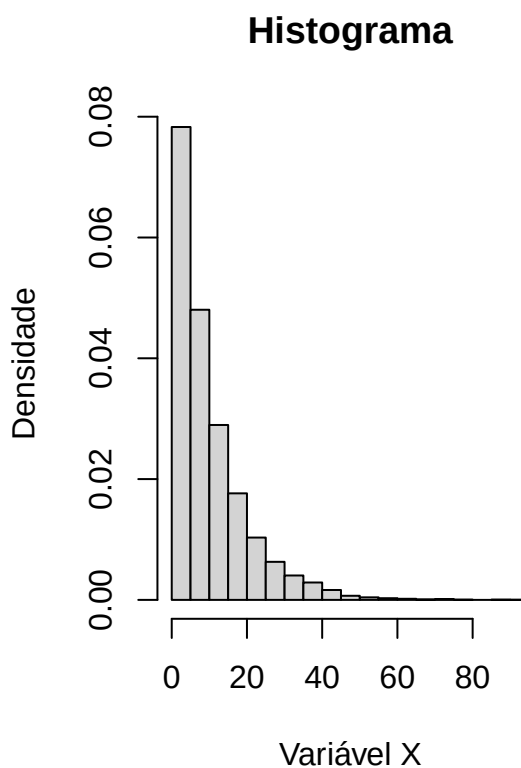
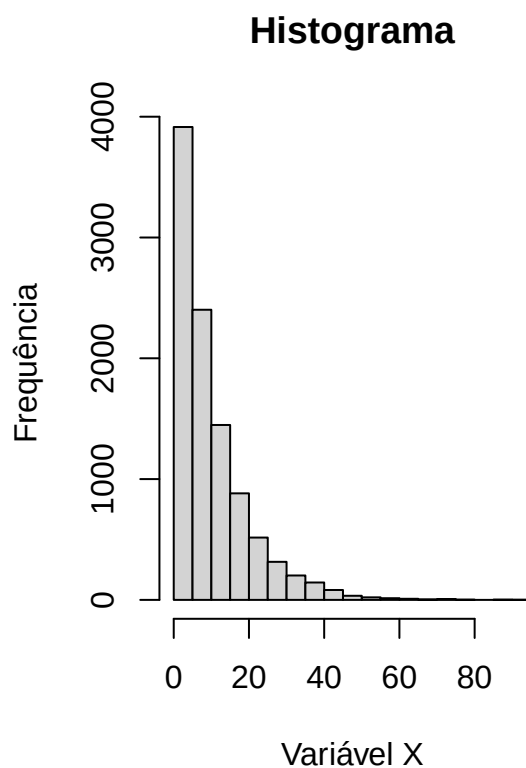
```
## [1] 29.95732
```

e) Geração de valores simulados e histograma

```
set.seed(123)

# Geração de 10000 valores aleatórios da distribuição exponencial com parâmetro beta = 0,1
x_sim = rexp(10000, beta)

# Construção de histogramas dos valores simulados
par(mfrow = c(1, 2))
hist(x_sim, xlab = "Variável X", ylab = "Frequência", main = "Histograma", freq = TRUE)
hist(x_sim, xlab = "Variável X", ylab = "Densidade", main = "Histograma", freq = FALSE, prob = TRUE)
```



f) Medidas de posição e dispersão

```
# Cálculo da média
mean(x_sim) # Aproximação numérica
```

```
## [1] 10.03781
```

```
1/beta # Valor exato
```

```
## [1] 10
```

```
# Cálculo da mediana
median(x_sim) # Aproximação numérica
```

```
## [1] 6.985061
```

```
qexp(0.50, beta) # Valor exato
```

```
## [1] 6.931472
```

```
# Cálculo da moda
# Observação: a distribuição exponencial tem densidade estritamente decrescente
# em  $x > 0$ , então a moda é sempre 0, independentemente de beta.
library(modeest) # Carrega o pacote para calcular a moda
expMode(beta)
```

```
## [1] 0
```

```
# Cálculo da variância
var(x_sim) # Aproximação numérica
```

```
## [1] 99.97644
```

```
1/beta^2 # Valor exato
```

```
## [1] 100
```

Exercício 6

A distribuição Beta é uma das distribuições de probabilidade contínuas mais populares definidas no intervalo $(0, 1)$. Ela é bastante flexível, pois, dependendo dos valores de seus parâmetros, pode assumir diferentes formatos — simétrica, assimétrica à esquerda, assimétrica à direita, ou até mesmo concentrada nos extremos.

Considere que: X é a proporção de votos de um determinado candidato, e suponha que os parâmetros da distribuição são $\alpha = 2$ e $\beta = 3$.

$$X \sim \text{Beta}(2, 3)$$

Função densidade de probabilidade:

$$p(X) = \frac{\Gamma(\alpha)\Gamma(\beta)}{\Gamma(\alpha + \beta)} x^{\alpha-1} (1-x)^{\beta-1}, \quad 0 < x < 1$$

em que $\alpha > 0$ e $\beta > 0$ são denominados parâmetros de forma.

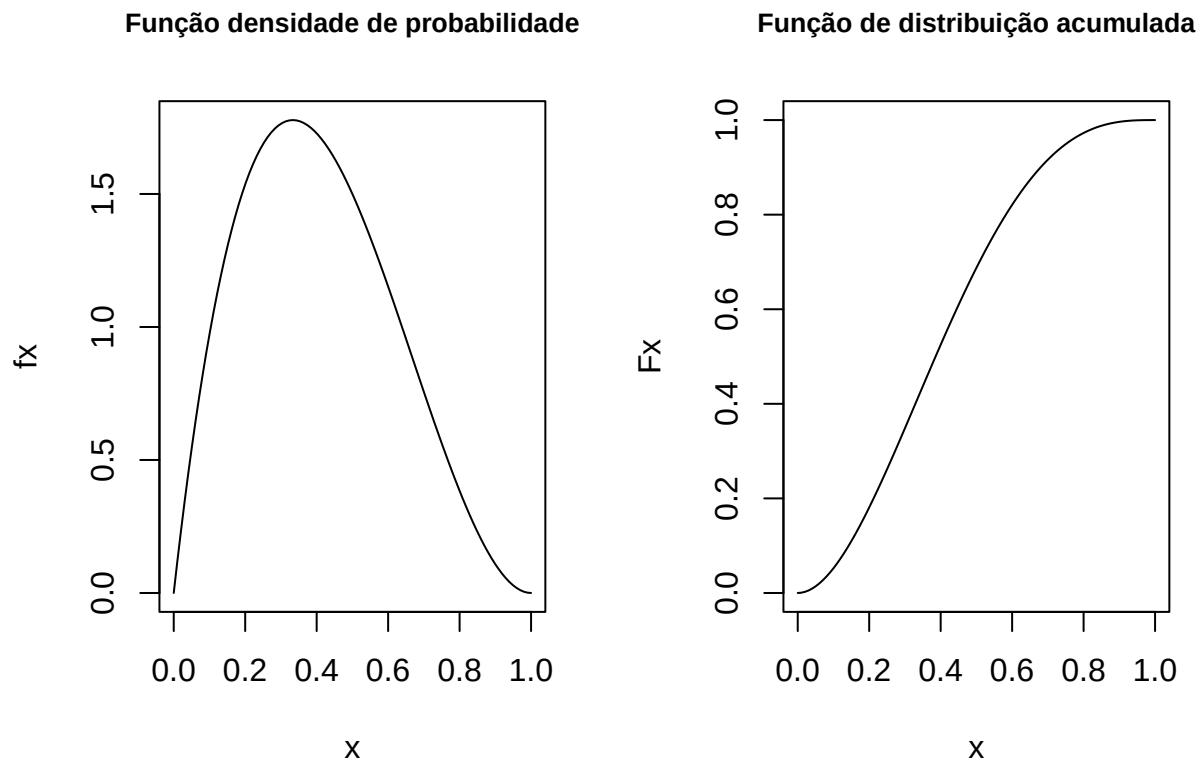
Parâmetro de forma

Parâmetro de forma é um tipo de parâmetro de uma distribuição de probabilidade que controla o formato da curva (a assimetria, a dispersão relativa, o achatamento etc.), sem necessariamente alterar a sua posição média ou a escala.

Gráficos

```
# Parâmetros da distribuição Beta
alfa = 2
beta = 3

# Gráfico da função densidade de probabilidade e da função de distribuição acumulada
par(mfrow = c(1, 2))
x <- seq(0, 1, length.out = 100)
fx <- dbeta(x, alfa, beta)
plot(x, fx, type = "l", main = "Função densidade de probabilidade", cex.main = 0.8)
Fx <- pbeta(x, alfa, beta)
plot(x, Fx, type = "l", main = "Função de distribuição acumulada", cex.main = 0.8)
```



Cálculo da Moda

Procedimento 1

```
# Cálculo da moda

# Procedimento 1: usando a função betaMode do pacote modeest
library(modeest) # Carrega o pacote para calcular a moda
betaMode(alfa, beta) # Calcula a moda (não aceita alfa ou beta < 1)
```

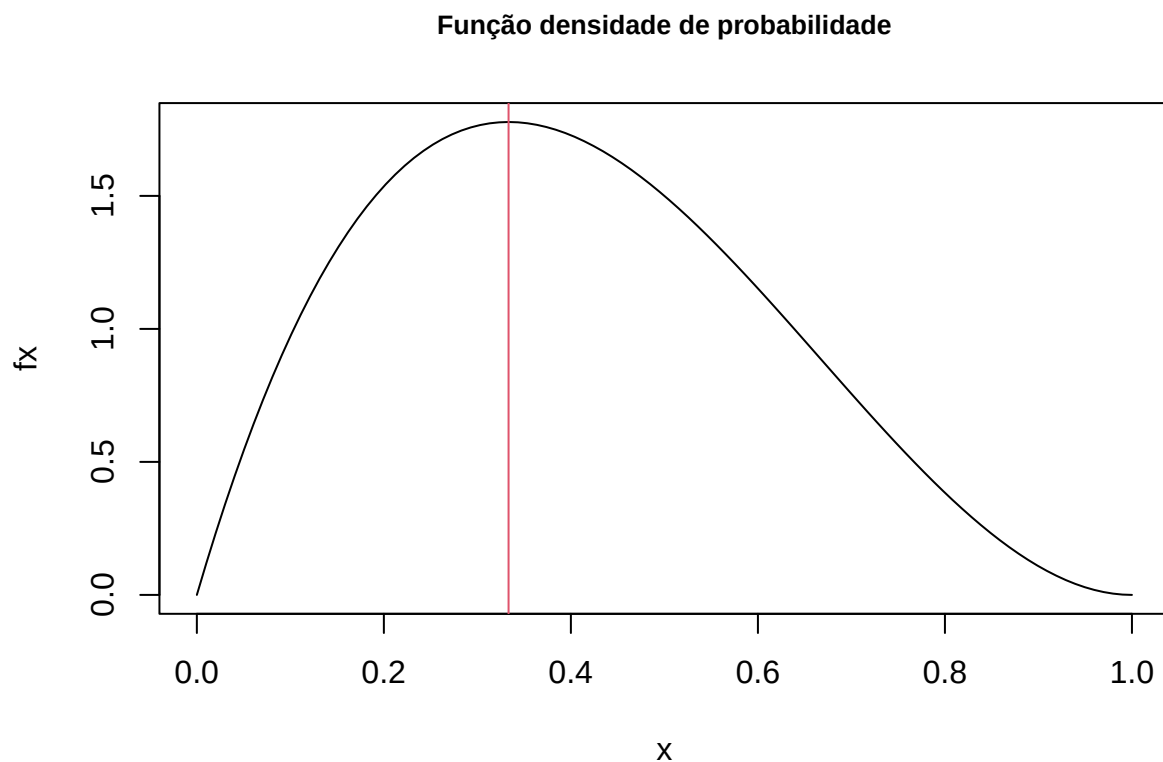
```
## [1] 0.3333333
```

Procedimento 2

```
# Procedimento 2: usando a função optimize para encontrar o ponto de máximo  
fbeta = (gamma(alfa) * gamma(beta)) / gamma(alfa + beta) # Função Beta  
f = function(x) (1/fbeta) * (x^(alfa-1)) * ((1-x)^(beta-1)) # Função densidade da Beta  
optimize(f, c(0, 1), maximum = TRUE) # Encontra o ponto de máximo
```

```
## $maximum  
## [1] 0.3333205  
##  
## $objective  
## [1] 1.777778
```

```
# Gráfico com o ponto de máximo (moda)  
x <- seq(0, 1, length.out = 100)  
fx <- dbeta(x, alfa, beta)  
plot(x, fx, type = "l", main = "Função densidade de probabilidade", cex.main = 0.8)  
M = betaMode(alfa, beta)  
abline(v = M, col = 2)
```



Efeito dos parâmetros

A ideia é ilustrar como a distribuição beta pode ser flexível, adaptando-se a diferentes formas de dados.

- a) $\alpha < 1$ e $\beta < 1 \rightarrow$ curva em forma de “U” (valores concentrados nos extremos 0 e 1).
- b) $\alpha < 1$ e $\beta > 1 \rightarrow$ curva concentrada próxima de 0.
- c) $\alpha > 1$ e $\beta < 1 \rightarrow$ curva concentrada próxima de 1.
- d) $\alpha > 1$ e $\beta > 1 \rightarrow$ curva em formato de sino (mais centralizada).
- e) $\alpha = 1$ e $\beta = 1 \rightarrow$ distribuição uniforme.

```
x <- seq(0, 1, length.out = 100)

# Dois gráficos por linha para os 4 primeiros casos
par(mfrow = c(3, 2), mar = c(4, 4, 2, 1))

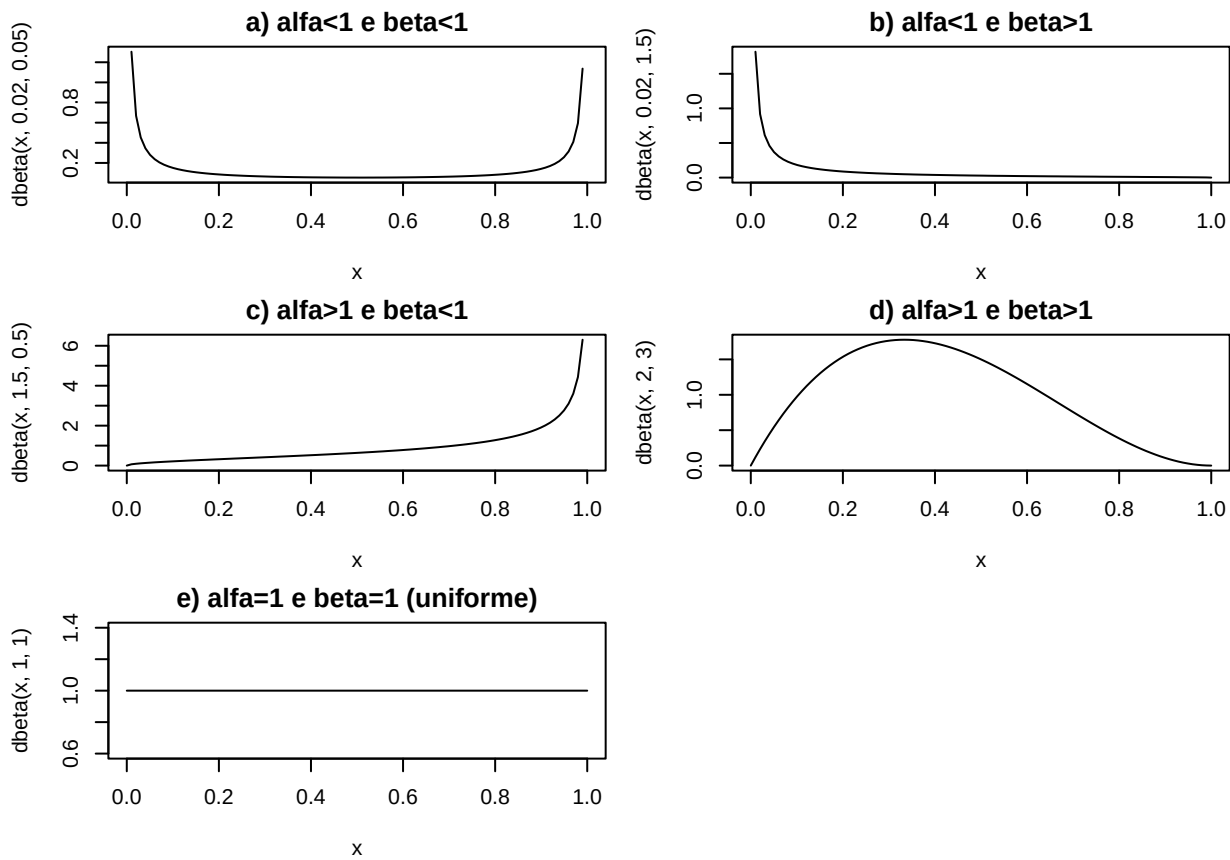
# a) alfa < 1 e beta < 1
plot(x, dbeta(x, 0.02, 0.05), type = "l", main = "a) alfa<1 e beta<1")

# b) alfa < 1 e beta > 1
plot(x, dbeta(x, 0.02, 1.5), type = "l", main = "b) alfa<1 e beta>1")

# c) alfa > 1 e beta < 1
plot(x, dbeta(x, 1.5, 0.5), type = "l", main = "c) alfa>1 e beta<1")

# d) alfa > 1 e beta > 1
plot(x, dbeta(x, 2, 3), type = "l", main = "d) alfa>1 e beta>1")

# e) alfa = 1 e beta = 1 (uniforme)
plot(x, dbeta(x, 1, 1), type = "l", main = "e) alfa=1 e beta=1 (uniforme)")
```



Exercício 7

Tempos de sobrevivência (em meses) de 44 pacientes com uma determinada doença:

```
t = c(12, 12, 4, 6, 17, 15, 3, 3, 4, 3, 9, 3, 2, 15, 5, 2, 20, 4, 8, 1, 5, 26, 12, 1, 10,
      2, 5, 11, 10, 7, 16, 1, 9, 9, 3, 5, 26, 9, 12, 25, 10, 9, 1)
```

O objetivo deste exercício é ajustar diferentes distribuições de probabilidade aos dados e verificar qual delas oferece o melhor ajuste.

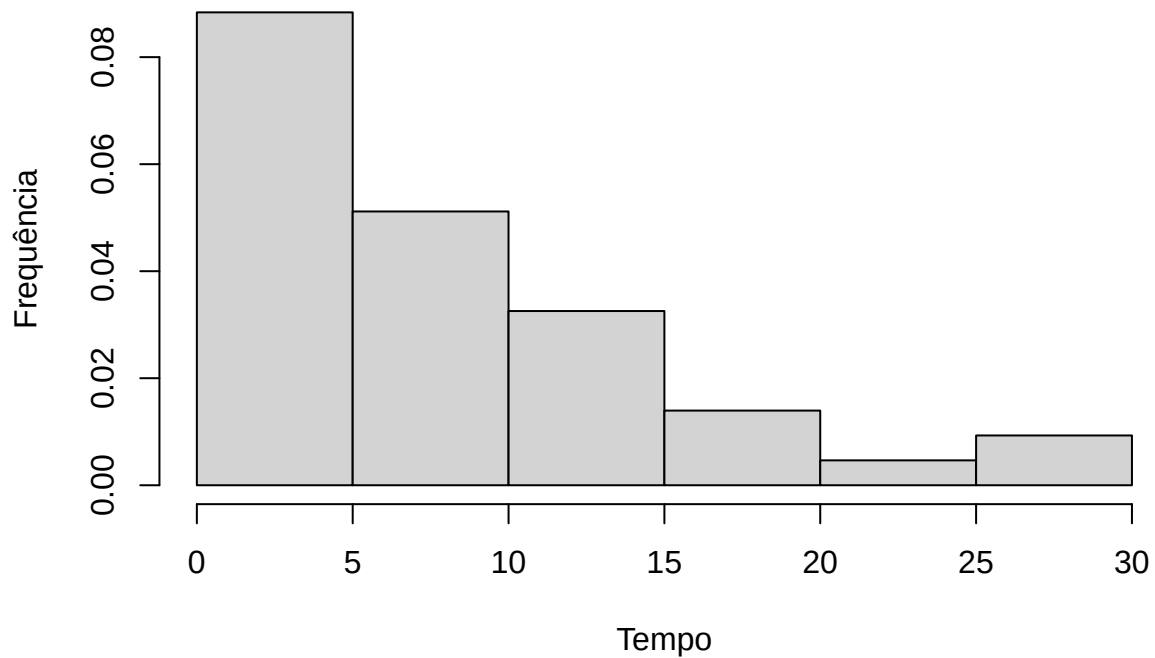
Passo 1: Ajuste das distribuições

Vamos ajustar os dados às distribuições Exponencial, Gama e Lognormal usando a função `fitdistr()` do pacote `MASS`.

```
# Dados de tempo de sobrevivência
t = c(12, 12, 4, 6, 17, 15, 3, 3, 4, 3, 9, 3, 2, 15, 5, 2,
      20, 4, 8, 1, 5, 26, 12, 1, 10, 2, 5, 11, 10, 7, 16,
      1, 9, 9, 3, 5, 26, 9, 12, 25, 10, 9, 1)

# Cria um histograma dos dados observados
hist(t, xlab = "Tempo", ylab = "Frequência", main = "Histograma de tempo de sobrevivência",
     freq = FALSE)
```

Histograma de tempo de sobrevivência



```
# Ajuste dos dados a diferentes distribuições
library(MASS) # Carrega o pacote MASS para ajuste de distribuições

fit_exp=fitdistr(t, "exponential") # Ajuste à distribuição exponencial
fit_exp
```

```
##      rate
## 0.11559140
## (0.01762752)
```

```
fit_gamma=fitdistr(t, "gamma") # Ajuste à distribuição gama
fit_gamma
```

```
##      shape      rate
## 1.59797731 0.18471270
## (0.31519866) (0.04270734)
```

```
fit_log_normal=fitdistr(t, "lognormal") # Ajuste à distribuição lognormal
fit_log_normal
```

```
##      meanlog      sdlog
## 1.81326401 0.90198630
## (0.13755162) (0.09726368)
```

Passo 2: Gráfico do histograma com as densidades ajustadas

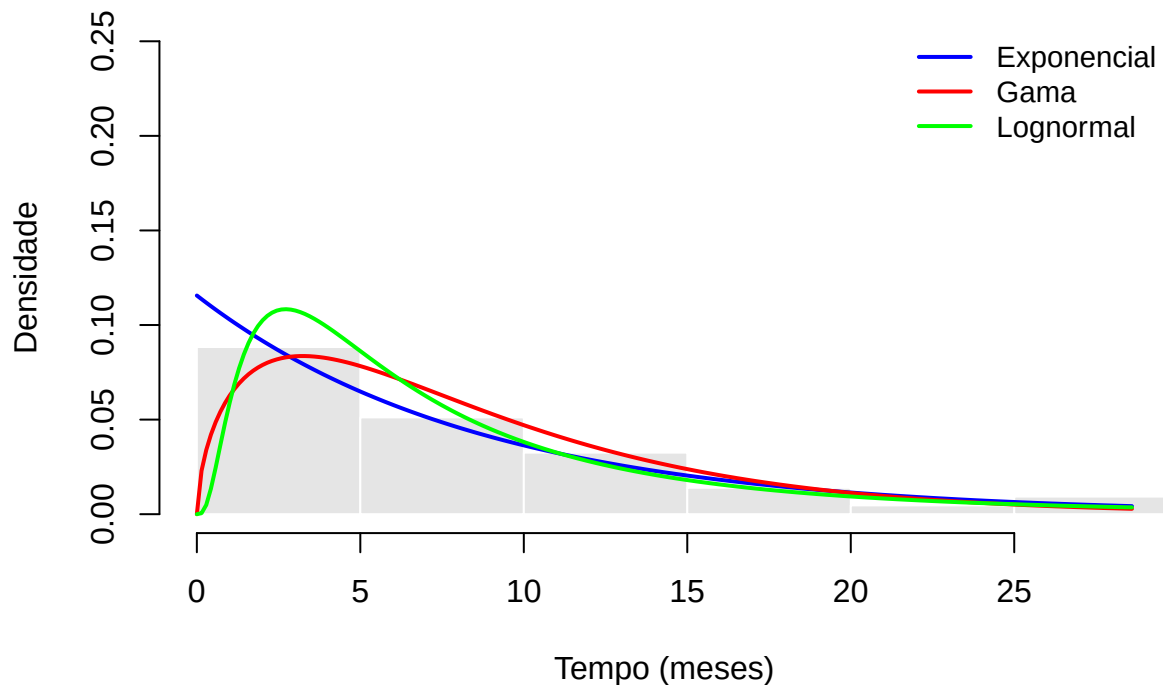
Para visualizar o ajuste, vamos plotar o histograma dos dados e sobrepor as funções densidade ajustadas. Para melhorar a visualização, criamos uma sequência `x_vals` que cobre toda a faixa dos dados com uma pequena margem e ajustamos os limites dos eixos.

```
# Sequência para avaliação das densidades
x_vals <- seq(0, max(t) * 1.1, length.out = 200)

# Plot do histograma
hist(t, xlab = "Tempo (meses)", ylab = "Densidade", main = "",
     freq = FALSE, prob = TRUE,
     xlim = c(0, max(t) * 1.1), ylim = c(0, 0.25), col = "gray90", border = "white")

# Sobreposição das curvas de densidade ajustadas
lines(x_vals, dexp(x_vals, rate = fit_exp$estimate), col = "blue", lwd = 2)
lines(x_vals, dgamma(x_vals, shape = fit_gamma$estimate[1],
                    rate = fit_gamma$estimate[2]), col = "red", lwd = 2)
lines(x_vals, dlnorm(x_vals, meanlog = fit_log_normal$estimate[1],
                    sdlog = fit_log_normal$estimate[2]), col = "green", lwd = 2)

legend("topright", bty = "n", cex = 0.9,
      col = c("blue", "red", "green"), lwd = 2,
      legend = c("Exponencial", "Gama", "Lognormal"))
```



Passo 3: Testes de aderência (Kolmogorov-Smirnov)

Para avaliar estatisticamente o ajuste das distribuições, realizamos o teste de Kolmogorov-Smirnov (KS) usando os parâmetros **estimados no Passo 1** (`fit_exp`, `fit_gamma`, `fit_log_normal`) — e não valores arbitrários, para que o teste avalie de fato o ajuste que acabamos de fazer.

Interpretação: valores de p-valor maiores que 0,05 indicam que não rejeitamos a hipótese de que os dados seguem aquela distribuição.

Nota técnica: como os parâmetros foram estimados a partir dos próprios dados `t`, o p-valor do teste KS clássico não é exatamente válido (a distribuição nula assume parâmetros conhecidos a priori, não estimados). Isso tende a tornar o teste conservador (mais difícil rejeitar H_0). Para uma inferência mais rigorosa seria melhor usar um teste com correção (ex.: teste de Lilliefors, `nortest::lillie.test` para o caso normal, ou bootstrap paramétrico); aqui usamos o KS “ingênuo” apenas como comparação exploratória entre os três ajustes.

```
# Hipótese nula: os dados seguem a distribuição exponencial (parâmetro estimado)
ks.test(t, "pexp", rate = fit_exp$estimate)

##
## Asymptotic one-sample Kolmogorov-Smirnov test
##
## data: t
## D = 0.13503, p-value = 0.4131
## alternative hypothesis: two-sided

# Hipótese nula: os dados seguem a distribuição gama (parâmetros estimados)
ks.test(t, "pgamma", shape = fit_gamma$estimate["shape"], rate = fit_gamma$estimate["rate"])

##
## Asymptotic one-sample Kolmogorov-Smirnov test
##
## data: t
## D = 0.11234, p-value = 0.6496
## alternative hypothesis: two-sided

# Hipótese nula: os dados seguem a distribuição lognormal (parâmetros estimados)
ks.test(t, "plnorm", meanlog = fit_log_normal$estimate["meanlog"],
        sdlog = fit_log_normal$estimate["sdlog"])

##
## Asymptotic one-sample Kolmogorov-Smirnov test
##
## data: t
## D = 0.1532, p-value = 0.2651
## alternative hypothesis: two-sided
```

Passo 4: Simulação e visualização de ajustes exponenciais para diferentes tamanhos amostrais

Para entender o efeito do tamanho da amostra na qualidade do ajuste, simulamos dados exponenciais e mostramos seus histogramas com a densidade ajustada para amostras de tamanhos variados.

```

set.seed(123)

# Função para ajustar e plotar a distribuição exponencial
ajustar_exponencial <- function(n) {
  # Gera dados simulados a partir de uma distribuição exponencial com taxa (rate) = 1
  dados <- rexp(n, rate = 1)

  # Ajusta a distribuição exponencial aos dados
  ajuste <- fitdistr(dados, "exponential")

  # Cria um histograma dos dados simulados
  hist(dados, xlab = "Dados", ylab = "Frequência", main = paste("Histograma - n =", n),
       freq = FALSE, xlim = c(0, max(dados)))

  # Adiciona a função de densidade ajustada
  curva <- function(x) dexp(x, rate = ajuste$estimate)
  curve(curva, add = TRUE, col = "blue", lwd = 2)

  # Adiciona a legenda
  legend("topright", legend = paste("Rate =", round(ajuste$estimate, 2)),
       col = "blue", lwd = 2)
}

# Configura a área de gráficos para exibir os histogramas lado a lado
par(mfrow = c(1, 3)) # 1 linha e 3 colunas

# Ajusta e plota a distribuição exponencial para diferentes tamanhos de amostra
ajustar_exponencial(10)
ajustar_exponencial(100)
ajustar_exponencial(100000)

```

